

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Answer:

- a) Incorrect/invalid field: "Content-Length: -150" is invalid, since Content-Length must always be a non-negative integer representing the number of bytes in the response body. A negative value has no real meaning in byte terms.
- b) Impact of the Content-Length issue: Since the client relies on Content-Length to know exactly how many bytes of body to read, an invalid negative value means the browser/proxy cannot determine where the response ends. This can cause the

connection to hang, the response to be truncated or rejected as malformed, and can break persistent (keep-alive) connections because the boundary between this response and the next is lost.

c) Status code correctness: The status line "HTTP/1.1 200 OK" indicates a successful request, but a response cannot really be considered fully correct when one of its headers is invalid. So while the status code itself is a valid code, the overall response is not correctly formed because the Content-Length contradicts basic HTTP rules.

d) Suggested correction: Set Content-Length to the actual byte size of the body, for example "Content-Length: 150" (a positive number matching the real content size), or remove Content-Length altogether and instead use "Transfer-Encoding: chunked" if the body size isn't known in advance.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

Answer:

a) Missing/improperly closed elements: The `<p>` element that contains "Hello World" is opened but never explicitly closed with `</p>`. The `</div>` tag then appears directly after it, meaning the paragraph and div close in the wrong, overlapping order.

b) How the browser will interpret this: HTML5 parsers apply an auto-closing rule for `<p>` — when a block-level element such as `<div>` begins (or in this case, the parser reaches the closing `</div>`), any open `<p>` is implicitly closed first. So the browser will silently insert a closing `</p>` right before `</div>` even though it isn't written in the source.

c) Impact on rendering and layout: Visually the page will usually still look correct because the browser repairs the structure automatically. However, the actual DOM tree the browser builds may not match what the developer intended — for example, if more elements were meant to sit inside that `<p>`, they would end up outside it instead, which can break CSS styling or JavaScript that expects a specific nesting.

d) Corrected DOM structure:

```

<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>

```

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

Answer:

a) Difference in data transmission: GET sends data as name=value pairs appended to the URL's query string (e.g. /login?user=admin&pass=1234), so it is fully visible in the address bar, browser history, and server logs. POST instead places the data inside the request body, so it is not shown in the URL and is not stored in browser history.

b) Which method is more suitable for login: POST is more suitable, because credentials are kept out of the URL, browser history, bookmarks, and typical server access logs, and it also supports larger and more structured payloads than GET's URL-length limits allow.

c) Security risks of using GET for authentication: Passwords sent via GET can be stored in browser history and cache, logged in plaintext by web servers and proxies, exposed through the Referer header to third-party sites, and easily leaked if a user copies or shares the URL — all of which make credential theft much easier.

d) Best practices for secure login: Always use POST over HTTPS (TLS) so data in transit is encrypted, never store or log raw passwords (use salted hashing such as bcrypt on the server), use session tokens or JWTs after authentication rather than resending credentials, add CSRF protection, implement rate-limiting/account lockout to resist brute-force attempts, and validate/sanitize all input on the server.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

Answer:

a) Missing closing tags: The `</h1>` tag after "Welcome" and the `</p>` tag after "This is a paragraph" are both missing.

b) Issues with the `<img>` tag: The tag is missing the "alt" attribute, which is required for accessibility (screen readers) and is shown if the image fails to load. In stricter XHTML syntax it would also need to be self-closed as ``, though in HTML5 `<img>` is a void element and does not require a closing tag.

c) How browsers will handle these errors: Browsers apply the same implicit closing rules seen earlier — when a new block-level element such as `<p>` appears, any open `<h1>` is automatically closed first, and similarly the open `<p>` is closed once a following block-level context ends the paragraph. The page will typically still render close to what was intended, but the actual parsed DOM structure differs from the raw source, which can cause unexpected styling or scripting issues.

d) Corrected version:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>

</body>
</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Answer:

a) Interpreting each step: (1) The user types a URL into the browser. (2) The browser performs a DNS lookup to translate the domain name into an IP address. (3) The browser opens a connection (TCP, plus a TLS handshake for HTTPS) and sends an HTTP request to that IP address. (4) The server receives the request, processes it (e.g., running application logic, querying a database), and prepares a response. (5) The server sends the HTTP response back to the browser. (6) The browser parses the returned HTML/CSS/JavaScript and renders the final page.

b) Where delays can occur: A slow or uncached DNS lookup, the TCP/TLS handshake overhead, slow server-side processing (e.g., heavy database queries or business logic), network latency and limited bandwidth while transferring the response, and the browser's own parsing/rendering time for large HTML, CSS, JavaScript, or images.

c) Role of DNS in the process: DNS acts as the internet's directory service, converting the human-readable domain name into the numeric IP address the browser needs before it can even open a connection to the server. If this step is slow (for example, an uncached lookup through multiple DNS servers), it delays everything that follows, since no request can be sent until the IP address is known.

d) Suggested optimizations: Use DNS caching and prefetching, serve content through a CDN to reduce network distance, reuse connections with HTTP keep-alive and take advantage of HTTP/2 or HTTP/3 multiplexing, cache responses on the server and browser side, compress responses (gzip/Brotli), and minify/optimize front-end assets (HTML, CSS, JS, images) to speed up rendering.

Code of Conduct:

I (K.sai kumar / 192372080) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.sai kumar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand